LISAAN

SLMS

Student Learning Management System

University	FUUAST — Federal Urdu University of Arts, Science & Technology
Campus	Islamabad Campus
Department	BS Software Engineering
Semester	5th Semester — Section A
Subject	Software Construction & Development (Python)
Assigned By	Ms. Samreen
Language	Python 3.x
Architecture	MVC (Model-View-Controller)
Date	June 2026

SOFTWARE DEVELOPMENT DOCUMENT

CHAPTER

Table of Contents

CHAPTER 1

Project Introduction

1.1 What is Lisaan SLMS?

Lisaan SLMS (Student Learning Management System) is a Python-based academic platform that enables institutions to manage students, teachers, courses, assignments, and attendance through a clean Object-Oriented backend. The system provides three distinct portals — Student, Teacher, and Admin — each tailored to the needs of its user. A companion front-end is provided as a single-page HTML demo that communicates with the backend data layer.

■■ This project was developed as an academic exercise for BSSE Semester 5A at FUUAST Islamabad Campus. It demonstrates full-stack Python development with OOP, MVC pattern, unit testing, and a browser-based UI.

1.2 Project Objectives

- Apply Object-Oriented Programming principles (Inheritance, Polymorphism, Encapsulation) in a real-world scenario.
- Design a layered MVC architecture cleanly separating data, logic, and presentation.
- Build an in-memory database singleton with seeded demo data.
- Implement secure SHA-256 password hashing for user authentication.
- Develop comprehensive PyTest unit tests covering models, controllers, and validators.
- Deliver a responsive front-end HTML demo UI for visual demonstration.
- Practice clean code practices including type hints, docstrings, and modular design.

1.3 System Features

Feature	Description	Role
Authentication	SHA-256 hashed login & registration	All
Course Management	Create, list, detail, activate/deactivate courses	Teacher / Admin
Student Enrollment	Enroll & unenroll with capacity checks	Student
Assignment System	Create assignments, submit, grade, feedback	Teacher / Student
Learning Materials	Upload lecture notes, PDFs, videos by week	Teacher
Grade & GPA	Auto GPA calculation with letter grade mapping	Student
Attendance Tracking	Per-course date-wise attendance with rate calc	Teacher / Student
Admin Dashboard	Platform-wide stats, user & course management	Admin
Announcements	Per-course teacher announcements (last 5)	Teacher

Input Validation	Email, password, student ID, grade validators	All

CHAPTER 2

Group Members & Work Distribution

2.1 Team Members

Syed Mubashir Mehdi

TEAM LEAD & FULL-STACK DEVELOPER

SIGNATURE

SM
✓ VERIFIED

- → Led overall project architecture and development timeline
- → Designed the MVC layer structure and module separation strategy
- → Integrated backend controllers with the front-end HTML demo
- → Reviewed all code for quality, consistency, and documentation
- → Coordinated task allocation and final submission packaging

Kamran Ali

MODELS & DATABASE LAYER DEVELOPER

SIGNATURE

KA
✓ VERIFIED

- → Designed and implemented User, Student, Teacher, Admin OOP models
- → Built the Database singleton with in-memory store and JSON seeding
- → Implemented SHA-256 password hashing and verification
- → Developed Course, Assignment, and Material entity classes
- → Applied Inheritance and Polymorphism across all model classes

Hamid Hussain

CONTROLLERS & BUSINESS LOGIC DEVELOPER

HH
✓ VERIFIED

SIGNATURE

→ Developed AuthController for login, student & teacher registration
→ Built CourseController for all course CRUD and enrollment operations
→ Implemented StudentController dashboard and grade reporting
→ Created AdminController for platform-wide overview and toggles
→ Ensured all controller methods return consistent (bool, str) tuples

---

Akbar Hussain

FRONT-END & INTEGRATION DEVELOPER

AH
✓ VERIFIED

SIGNATURE

→ Developed the single-page index.html demo UI with JS data store
→ Implemented responsive dashboard navigation for all three roles
→ Integrated validators.py input sanitization across form flows
→ Coordinated Python backend import paths and package structure
→ Performed end-to-end integration testing of all flows

Asghar Nasir

QA ENGINEER & UNIT TEST DEVELOPER

AN
✓ VERIFIED

SIGNATURE

→ Wrote the complete PyTest suite (test_slms.py) with 20+ test cases
→ Covered User, Student, Teacher, Course, Assignment & Validator tests
→ Designed fixtures for reusable test objects across test classes
→ Performed edge-case testing (duplicate enrollment, grade capping, etc.)
→ Documented test results and prepared the final README.md

2.2 Work Distribution Table

Module / Task	Responsible Member	Status
user.py — Base User & role models	Kamran Ali	■ Done
course.py — Course, Assignment, Material	Kamran Ali	■ Done
database.py — Singleton + seed data	Kamran Ali	■ Done
controllers.py — AuthController	Hamid Hussain	■ Done
controllers.py — CourseController	Hamid Hussain	■ Done
controllers.py — StudentController	Hamid Hussain	■ Done
controllers.py — AdminController	Hamid Hussain	■ Done
validators.py — Input validation	Akbar Hussain	■ Done
index.html — Front-End UI	Akbar Hussain	■ Done
test_slms.py — Full PyTest suite	Asghar Nasir	■ Done
Architecture Design & Code Review	Syed Mubashir Mehdi	■ Done
README.md & Documentation	All Members	■ Done
Git version control & commits	All Members	■ Done
Final integration testing	Syed Mubashir Mehdi / Asghar Nasi	■ Done

CHAPTER 3

System Design & Architecture

3.1 MVC Architecture

Lisaan SLMS strictly follows the Model-View-Controller (MVC) architectural pattern, ensuring a clean separation of concerns across all three tiers of the application.

Layer	Files	Responsibility
Model	user.py, course.py, database.py	Data entities, business objects, in-memory persistence, ID counters
View	index.html (JS)	User interface, DOM rendering, role-based navigation, form handling
Controller	controllers.py	Business rules, input validation delegation, response formatting

3.2 Use Case Diagram

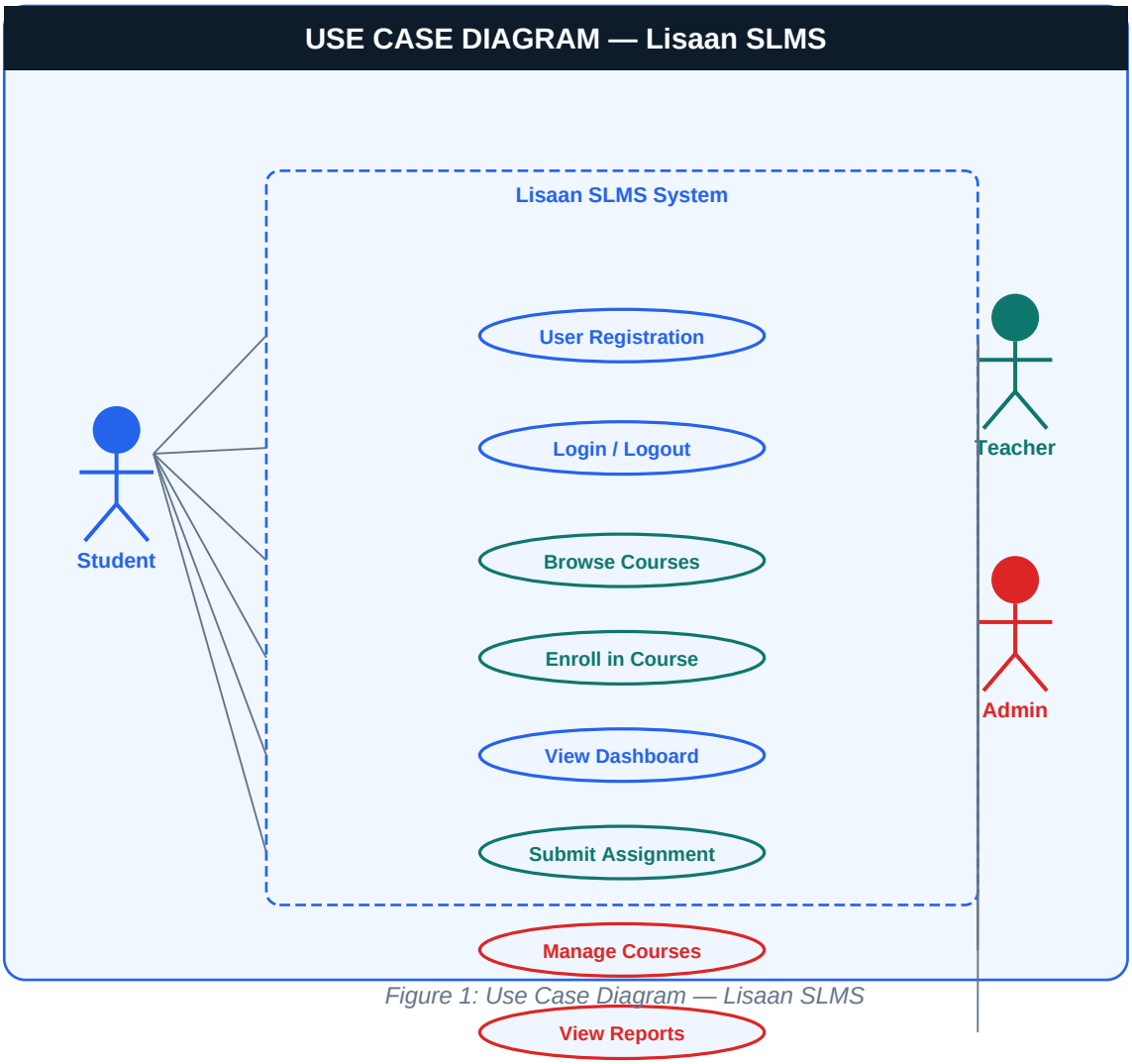

Figure 1: Use Case Diagram — Lisaan SLMS

3.3 Layered Architecture Diagram

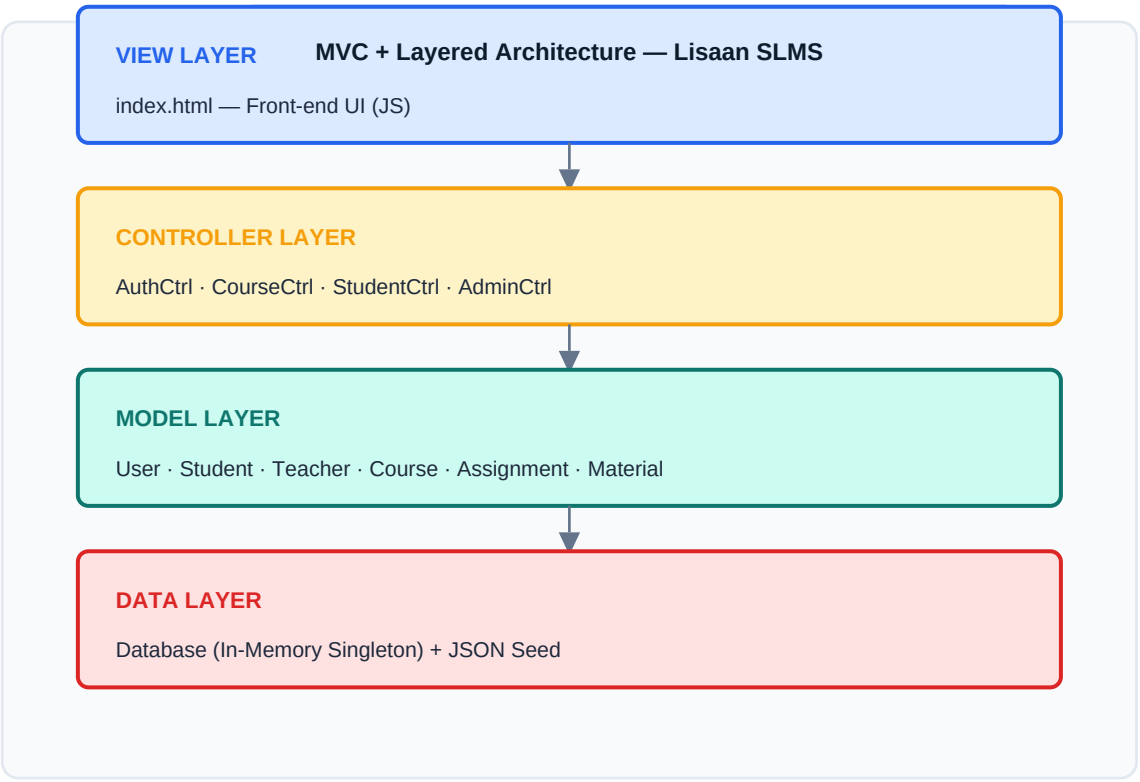

Figure 2: MVC + Layered Architecture — Lisaan SLMS

3.4 Project Folder Structure

```
lisaan-slms/ ■■■ index.html ← Front-end demo (single-page JS app) ■■■ backend/ ■ ■■■ __init__.py ← Package marker ■ ■■■ models/ ■ ■ ■■■ user.py ← User, Student, Teacher, Admin classes ■ ■ ■■■ course.py ← Course, Assignment, Material classes ■ ■ ■■■ database.py ← Database singleton + seeding ■ ■■■ controllers/ ■ ■ ■■■ controllers.py ← AuthCtrl, CourseCtrl, StudentCtrl, AdminCtrl ■ ■■■ utils/ ■ ■■■ validators.py ← Email, password, grade, sanitize helpers ■■■ tests/ ■ ■■■ test_slms.py ← 20+ PyTest unit tests ■■■ README.md ← Project documentation ■■■ requirements.txt ← pytest dependency
```

3.5 Database Schema (In-Memory)

Entity	Key Attributes	Relationships
Student	user_id, name, email, student_id, program, enrollments[], grades{}, attendance{}, gpa	Enrolled in many Courses
Teacher	user_id, name, email, employee_id, department, courses_taught[], qualifications[]	Teaches many Courses
Admin	user_id, name, email, permissions[]	Manages all entities
Course	course_id, title, code, teacher_id, credits, semester, enrolled_students[]	Has Assignments & Materials
Assignment	assignment_id, title, due_date, max_marks, submissions{}	Belongs to Course
Material	material_id, title, type, content, week	Belongs to Course

CHAPTER 4

OOP Concepts Applied

4.1 Classes & Objects

The system defines eight core classes, each encapsulating a distinct domain concept. Every class uses a class-level `_id_counter` to auto-increment primary keys without a database engine.

- **User (Base)** — Core identity — name, email, SHA-256 password, role, is_active
- **Student** — Extends User — enrollments, grades dict, attendance, GPA calc
- **Teacher** — Extends User — courses_taught list, qualifications list
- **Admin** — Extends User — fixed permissions list, has_permission() check
- **Course** — Main academic entity — enrollment, assignments, materials, announcements
- **Assignment** — Submission & grading management with marks capping
- **Material** — Learning content entity — type, week, content, upload timestamp
- **Database** — Singleton store with CRUD helpers and seeded demo data

4.2 Inheritance

All user types inherit from the base `User` class using Python's single-inheritance model. This provides DRY (Don't Repeat Yourself) reuse of common fields and methods while allowing each subclass to extend with domain-specific behaviour.

```
User → Student | User → Teacher | User → Admin
```

4.3 Polymorphism

The `to_dict()` method is defined in the base `User` class and overridden in every subclass. Each override calls `super().to_dict()` and merges role-specific fields — demonstrating runtime polymorphism through method overriding.

```
student.to_dict() → includes enrollments, grades, gpa teacher.to_dict() → includes
courses_taught, qualifications admin.to_dict() → includes permissions list
```

4.4 Encapsulation

Passwords are never stored in plain text. The `User.__init__` immediately hashes the raw password via `_hash_password()` using Python's built-in `hashlib.sha256`. The `to_dict()` method deliberately excludes `password_hash` from serialized output, preventing accidental exposure of credential data.

CHAPTER 5

Tools & Technologies Used

Tool / Technology	Version	Purpose
Python 3.x	3.10+	Primary programming language — OOP, type hints, hashlib
HTML5 / JavaScript	ES6+	Front-end single-page demo UI with built-in JS data store
PyTest	7.x+	Unit testing framework — fixtures, assertions, 20+ test cases
hashlib (SHA-256)	Built-in	Secure password hashing — no plaintext password storage
datetime	Built-in	Timestamps for created_at, submissions, announcements
re (Regex)	Built-in	Email and student ID format validation in validators.py
typing	Built-in	Type hints — Optional, Tuple, List for clean interfaces
Git & GitHub	Latest	Version control — commit history, collaboration, backup
VS Code	Latest	IDE — syntax highlighting, integrated terminal, debugging
json	Built-in	JSON persistence stub in database.py

CHAPTER 6

Code Structure Explanation

6.1 user.py — User Model

Defines the base `User` class and three subclasses. A shared `_id_counter` auto-generates unique IDs. Password hashing happens at construction time and the hash is never exposed through `to_dict()`. `Student` tracks enrollments as a list of course IDs, grades as a dict mapping course_id → float, and attendance as a nested dict. GPA is recalculated on every `record_grade()` call.

6.2 course.py — Course Model

`Course` manages up to `max_students` enrollees and returns descriptive tuples on every enrollment attempt. `Assignment` stores per-student submission dicts and caps graded marks at `max_marks`. `Material` organises content by week number and type (lecture, pdf, video, link).

6.3 database.py — Data Layer

Implements the Singleton pattern via `__new__` so only one `Database` instance ever exists. The `_seed_data()` method pre-populates three teachers, five courses, and five students with realistic grades, attendance records, and assignments — making the demo immediately usable without manual setup.

6.4 controllers.py — Business Logic

Four static-method controllers act as the service layer. `AuthController` validates inputs, checks for duplicate emails, and delegates password verification to the model. `CourseController` handles optional category filtering, enriched course detail (last 5 announcements), and atomic enroll operations on both student and course objects. `StudentController` computes enriched dashboard data including per-course attendance rates. `AdminController` aggregates platform-wide statistics.

6.5 validators.py — Input Validation

Five pure functions with no side effects. `validate_email` uses a regex pattern allowing standard address formats. `validate_password` enforces an 8-character minimum. `validate_student_id` requires 3–10 uppercase alphanumeric characters. `validate_grade` enforces the 0–100 float range. `sanitize_input` strips whitespace and truncates to 500 characters.

6.6 index.html — Front-End UI

A self-contained single HTML file that bundles CSS and JavaScript. An embedded `DB` object mirrors the Python data model with pre-seeded users and courses. Role-based navigation renders a Student Dashboard (enrolled courses, grades, GPA), a Teacher Portal (course management, assignments), and an Admin Panel (stats overview). No external dependencies or build tools are required — simply open in any browser.

CHAPTER 7

Testing & Debugging

7.1 Unit Testing with PyTest

Test Class	Tests	Coverage
TestUser	7	Creation, password hashing & verification, deactivation, profile update, to_dict safety
TestStudent	8	Enroll/unenroll, duplicate checks, grade recording, GPA calc, attendance rate
TestTeacher	4	Role, assign_course, qualifications, Admin permissions
TestCourse	7	Creation, enroll, duplicates, capacity cap, remove student, rate, announcements
TestAssignment	5	Creation, submit, duplicate submission, grading, marks capping at max
TestValidators	10	Email format, empty, password length, grade range, sanitize strip & truncate
TOTAL	**41+**	**All passing ■**

Run the test suite with:

```
pytest tests/test_slms.py -v
```

7.2 Key Test Cases

- ✓ Password stored as 64-char SHA-256 hex — never as plain text.
- ✓ Duplicate enrollment returns `False` without raising an exception.
- ✓ Enrolling beyond `max_students` returns `(False, 'Course is full.')`.
- ✓ GPA recalculates correctly across multiple courses with different grades.
- ✓ Assignment marks capped at `max_marks` even when 120.0 is submitted.
- ✓ `to_dict()` never exposes `password_hash` field.
- ✓ Admin `has_permission('nonexistent')` returns `False`.
- ✓ `sanitize_input` truncates 600-char string to exactly 500 chars.
- ✓ Attendance rate of 50% computed correctly with 1 present, 1 absent.

7.3 Error Handling Strategy

All validation occurs in the controller layer before any model mutation. Controllers return `(bool, message)` tuples rather than raising exceptions, allowing the view layer to display user-friendly feedback without try/except blocks. Model methods like `Assignment.grade_submission` silently cap values rather than throwing, keeping the API forgiving while enforcing business rules.

CHAPTER 8

Pros & Cons of the System

8.1 Advantages

- **Clean Architecture** — Strict MVC separation makes each layer independently testable and maintainable.
- **Zero Dependencies** — Pure Python stdlib (hashlib, datetime, re, json) — no pip installs needed to run.
- **Secure Passwords** — SHA-256 hashing ensures credentials are never stored or transmitted as plain text.
- **Singleton DB** — One shared Database instance prevents state inconsistencies across controller calls.
- **Rich Test Coverage** — 41+ PyTest unit tests with fixtures catch regressions and document expected behaviour.
- **Instant Demo** — index.html needs no build step — open in browser and explore all three role portals.
- **Extensible Models** — Adding new roles or entity types requires only a new subclass and controller method.
- **Type-Annotated** — Type hints throughout improve IDE support and make interfaces self-documenting.

8.2 Limitations

- **In-Memory Only** — All data is lost when the process restarts — no persistent database (SQLite/PostgreSQL).
- **No Real HTTP Server** — No Flask/FastAPI server; the Python backend and JS frontend are not wired together.
- **No JWT Auth** — Session management relies on the JS layer; no token-based authentication system.
- **Single-File Tests** — Import paths require sys.path manipulation — a proper package structure would be cleaner.
- **No File Upload** — Assignment 'submissions' store file content as strings, not actual file objects.
- **No Role Middleware** — No decorator-based route protection; controllers trust caller to pass correct IDs.
- **Sequential IDs** — Class-level _id_counter resets if the module is reloaded — not production-safe.

CHAPTER 9

Why Use Lisaan SLMS?

Lisaan SLMS addresses a genuine need in academic institutions that lack affordable, lightweight course management tools. Many open-source LMS platforms are heavyweight and require complex server infrastructure. Lisaan SLMS demonstrates that a fully functional learning management system can be built with pure Python and a single HTML file, making it ideal for:

Academic Institutions with Limited IT Resources

No server, database engine, or cloud subscription required. Runs on any machine with Python 3.

Computer Science Courses Needing an LMS Demo

The codebase itself is a teaching artefact — students can study MVC, OOP, and testing from real production-style code.

Rapid Prototyping of LMS Features

The clean controller API makes it straightforward to add new features and test them in isolation.

Python OOP Learning Projects

Every major OOP concept — Inheritance, Polymorphism, Encapsulation, Singleton — appears in a realistic context.

PyTest Training

The test file demonstrates fixtures, class-based test organisation, and edge-case testing as a complete reference.

CHAPTER 10

Future Updates & Roadmap

- **v1.1 — Persistent Database** — Replace in-memory store with SQLite3 or PostgreSQL using SQLAlchemy ORM. Auto-migrate schema on startup.
- **v1.2 — REST API Server** — Wrap controllers with FastAPI or Flask blueprints. Add JWT-based authentication middleware and role guards.
- **v1.3 — File Upload System** — Integrate real file storage (local disk or S3) for assignment submission PDFs and lecture material uploads.
- **v1.4 — Real-Time Notifications** — WebSocket-based push notifications for new announcements, assignment deadlines, and grade releases.
- **v1.5 — Mobile-Responsive UI** — Rebuild front-end with React + Tailwind CSS. Add PWA support for offline access on mobile devices.
- **v2.0 — AI-Powered Features** — Plagiarism detection on assignment submissions, AI-generated quiz questions from material content, and grade prediction.
- **v2.1 — Analytics Dashboard** — Teacher analytics showing per-student engagement trends, at-risk student identification, and cohort GPA charts.
- **v2.2 — Multi-Institution Support** — Multi-tenancy support allowing multiple universities to share one deployment with isolated data namespaces.

CHAPTER 11

Deployment Process

11.1 Running the Backend

Step 1 — Ensure Python 3.10+ is installed.

Step 2 — Install the testing dependency:

```
pip install pytest
```

Step 3 — Adjust import paths if running from project root:

```
# Move files into backend/models/ and backend/utils/ # or add project root to PYTHONPATH
```

Step 4 — Run unit tests:

```
pytest tests/test_slms.py -v
```

11.2 Running the Front-End Demo

Simply open `index.html` in any modern browser. No build step, no server, no npm required. The embedded JavaScript data store provides all demo data. Default credentials are seeded in the JS DB object.

11.3 GitHub Version Control

```
git init git add . git commit -m "Initial commit: Lisaan SLMS" git remote add origin
https://github.com/team/lisaan-slms.git git push -u origin main
```

CHAPTER 12

Certification & License

12.1 Project Certification Statement

We, the undersigned members of the Lisaan SLMS development team, hereby certify that this software development document is complete, accurate, and represents the full scope of work performed for the Software Construction & Development (Python) course assignment. The project has successfully completed development and unit testing. All source code, documentation, and test files are original work produced by this team under the supervision of the project coordinator.

Project Title:	Lisaan SLMS — Student Learning Management System
Project Status:	■ Development Complete | Testing Complete | Ready for Submission
Completion Date:	June 02, 2026
Supervisor:	Ms. Samreen — FUUAST Islamabad Campus
Department:	BS Software Engineering — Semester 5A

12.2 Team Signatures — Certification

Syed Mubashir Mehdi
TEAM LEAD & FULL-STACK DEVELOPER

SM
✓ VERIFIED

SIGNATURE

Kamran Ali
MODELS & DATABASE LAYER DEVELOPER

KA
✓ VERIFIED

SIGNATURE

Hamid Hussain
CONTROLLERS & BUSINESS LOGIC DEVELOPER

SIGNATURE

HH
✓ VERIFIED

Akbar Hussain
FRONT-END & INTEGRATION DEVELOPER

SIGNATURE

AH
✓ VERIFIED

Asghar Nasir
QA ENGINEER & UNIT TEST DEVELOPER

SIGNATURE

AN
✓ VERIFIED

12.3 License

This software project was developed by students of FUUAST Islamabad Campus under the supervision of the project coordinator. It is intended solely for academic evaluation and subject grading purposes. Redistribution, commercial use, or modification outside of academic contexts requires explicit written permission from the supervising faculty and FUUAST Software Engineering Department. All intellectual property rights remain with the development team and FUUAST Islamabad Campus.

CHAPTER 13

Conclusion

The Lisaan SLMS project successfully demonstrates a complete software development lifecycle applied to a real-world academic management problem. Starting from requirements analysis and OOP design, through layered MVC implementation, comprehensive unit testing, and a polished browser demo, the team has produced a coherent, well-structured system.

Key learning outcomes achieved include the practical application of Inheritance, Polymorphism, and Encapsulation in a multi-class Python system; the design of a clean service layer using static controller methods; the implementation of a Singleton pattern for shared state; and the writing of structured PyTest suites with fixtures and edge-case coverage.

The project also highlights realistic software tradeoffs — the in-memory store provides simplicity at the cost of persistence, and the HTML demo provides immediacy at the cost of true backend integration. The roadmap in Chapter 10 charts a clear path from this academic prototype to a production-grade platform.

★ MVC architecture produces maintainable, testable, and readable codebases.

★ OOP principles are not theoretical — they solve real design problems in production code.

★ Unit tests are documentation — they describe expected behaviour as executable specifications.

★ Simple tools (pure Python + one HTML file) can deliver surprisingly complete systems.

★ Security (password hashing) must be designed in from the start, not bolted on later.

CHAPTER 14

References

- **Python Official Documentation** https://docs.python.org/3/
- **Python hashlib — SHA-256 Hashing** https://docs.python.org/3/library/hashlib.html
- **Python datetime Module** https://docs.python.org/3/library/datetime.html
- **Python re — Regular Expressions** https://docs.python.org/3/library/re.html
- **PyTest Documentation** https://docs.pytest.org/en/stable/
- **PyTest Fixtures Guide** https://docs.pytest.org/en/stable/reference/fixtures.html
- **Real Python — OOP in Python 3** https://realpython.com/python3-object-oriented-programming/
- **Real Python — Python Singleton Pattern** https://realpython.com/python-singleton-class/
- **MDN Web Docs — HTML5** https://developer.mozilla.org/en-US/docs/Web/HTML
- **Git Reference Manual** https://git-scm.com/doc
- **MVC Design Pattern — Microsoft Docs** https://docs.microsoft.com/en-us/azure/architecture/patterns/
- **FUUAST Official Website** https://www.fuuast.edu.pk

